



第 7 章

Sorting

排序

Ajou University

内容

- 动机
- 插入排序(Insertion sort)
- 快速排序(Quick sort)
- 归并排序(Merge sort)
- 堆排序(Heap sort)

搜索与列表验证

❖ 为什么高效排序如此重要？

- 高效搜索
 - 顺序搜索
 - 二分搜索
- 高效列表验证
 - 比较列表以验证它们是否相同或不同
 - 使用一个列表中的每个键作为搜索键，重复搜索另一个列表

使用二分搜索会显著提高速度

二分搜索需要列表已排序
- 超过25%的计算时间用于排序
- 某些组织超过50%的计算时间用于排序列表

定义

❖ 给定一个记录列表 $(R_1, R_2, \dots, R_n)$

- 每个记录 R_i 具有一个键值 K_i
- 键值上存在一个序关系 ($<$)，对于任意两个键值 x 和 y ，满足以下条件之一： $x=y$, $x<y$, 或 $y < x$
- 该序关系具有传递性：

若 $x < y$ 且 $y < z$, 则 $x < z$

❖ 排序问题定义为

找到一个排列 σ ，使得对于 $1 \leq i \leq n-1$ ，满足 $K_{\sigma(i)} < K_{\sigma(i+1)}$ ， $1 \leq i \leq n-1$

❖ 稳定排序算法的定义，

若在输入列表中 $i < j$ 且 $K_i = K_j$ ，则在排序后的列表中， R_i 位于 R_j 之前。

排序分类

❖ 内部排序

- 列表规模较小，可完全在主存储器中进行排序。
- 待排序的信息能够存储在计算机内存中。

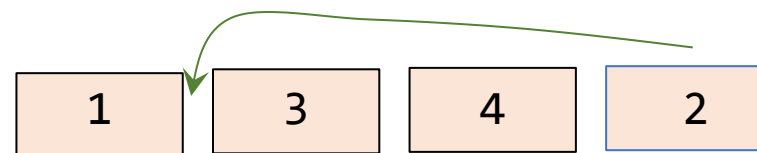
❖ 外部排序

- 排序过程中使用外部存储器（如磁带或磁盘）的排序算法。
- 当待排序的数据无法全部放入主存储器时使用。

插入排序

❖ 对于 j 从 1 到 $n-1$, 将 $\text{list}[j]$ 插入到已排序的子列表 $\text{list}[0], \dots, \text{list}[j-1]$ 中:

- 将所有大于 $\text{list}[j]$ 的键向右移动
- 时间复杂度
 - 最好情况: $O(n)$
 - 最坏情况: $O(n^2)$
- 若列表接近有序, 则工作量较小



❖ 乱序 (Left out of order: LOO)

- 记录 R_i 为乱序(LOO), 当且仅当 $R_i < \max_{1 \leq j < i} \{R_j\}$
- 插入步骤仅针对乱序的记录执行
- 计算乱序记录的数量 k_i
 - 计算时间: $O((k+1)n) = O(kn)$
 - 最坏情况时间: $O(n^2)$

插入排序代码

```
void insert(element e, element a[], int i)
{
    a[0] = e;
    while(e.key < a[i].key) {
        a[i+1] = a[i];
        i--;
    }
    a[i+1] = e;
}
```

```
void insertionSort(element a[], int n)
{
    int j;
    for(j = 2; j <= n; j++) {
        element temp = a[j];
        insert(temp, a, j-1);
    }
}
```

插入排序案例

- $n = 5$
- 输入序列: (26, 5, 37, 1, 19)
- LOOs = R_2, R_4, R_5

	[0]	[1]	[2]	[3]	[4]	[5]
		26	5	37	1	19
j=2	5	5	26	37	1	19
j=3	37	5	26	37	1	19
j=4	1	1	5	26	37	19
j=5	19	1	5	19	26	37

```
void insertionSort(element a[],int n)
{
    int j;
    for(j = 2; j <= n; j++) {
        element temp = a[j];
        insert(temp, a, j-1);
    }
}
```

```
void insert(element e, element a[], int i)
{
    a[0] = e;
    while(e.key < a[i].key) {
        a[i+1] = a[i];
        i--;
    }
    a[i+1] = e;
}
```


插入排序案例

- $n = 5$
- 输入序列: (37, 26, 19, 5, 1)
- LOOs = R_2, R_3, R_4, R_5

	[0]	[1]	[2]	[3]	[4]	[5]
		37	26	19	5	1
j=2	26	26	37	19	5	1
j=3	19	19	26	37	5	1
j=4	5	5	19	26	37	1
j=5	1	1	5	19	26	37

```
void insertionSort(element a[],int n)
{
    int j;
    for(j = 2; j <= n; j++) {
        element temp = a[j];
        insert(temp, a, j-1);
    }
}
```

```
void insert(element e, element a[], int i)
{
    a[0] = e;
    while(e.key < a[i].key) {
        a[i+1] = a[i];
        i--;
    }
    a[i+1] = e;
}
```

插入排序

时间复杂度

平均情况

[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
15	18	13	37	38	46	58	61

最好情况

$O(n)$

[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
3	5	13	35	37	46	58	61

最坏情况

$O(n^2)$

$1+2+3+...+n = n(n+1)/2$

[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
61	58	46	38	37	13	5	3

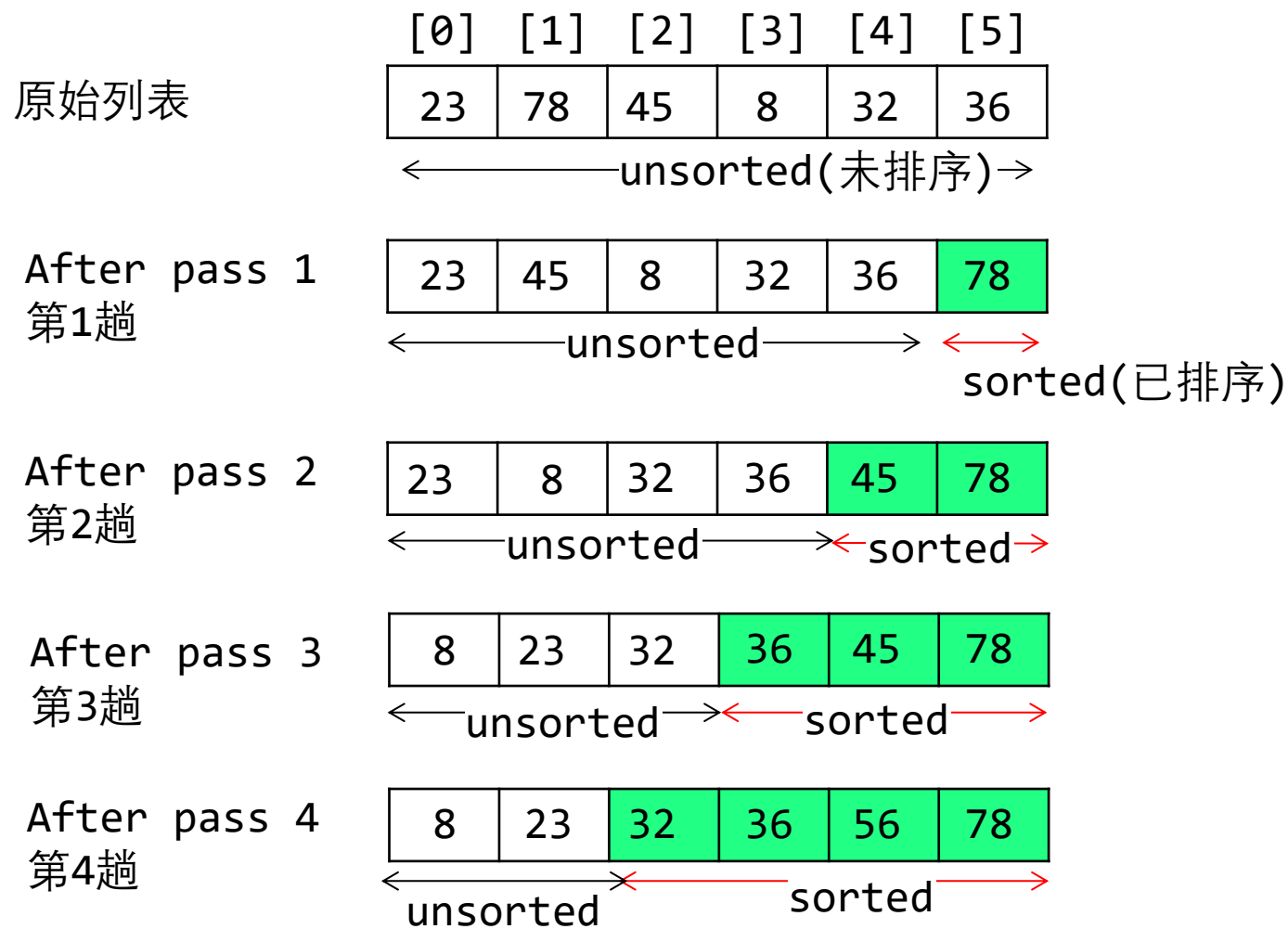
冒泡排序

- ❖ 最简单的对象数组排序方法
- ❖ 基本思想
 - 比较两个**相邻**对象
 - 若顺序错误则交换它们

冒泡排序代码

```
void bubbleSort (element a[], int n)
{
    element tmp; int i, j;
    for (i=0; i<n-1; i++) {
        for (j=0; j<n-1-i; j++)
            if (a[j+1] < a[j]) {
                tmp = a[j];
                a[j] = a[j+1];
                a[j+1] = tmp;
            }
    }
}
```

冒泡排序案例



冒泡排序案例

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	
5	26	37	1	61	11	59	15	i=0, j=0
5	26	37	1	61	11	59	15	i=0, j=1
5	26	1	37	61	11	59	15	i=0, j=2
5	26	1	37	61	11	59	15	i=0, j=3
5	26	1	37	11	61	59	15	i=0, j=4
5	26	1	37	11	59	61	15	i=0, j=5
5	26	1	37	11	59	15	61	i=0, j=6
5	26	1	37	11	59	15	61	i=1, j=0
	26	1						i=1, j=1
	1	26						

冒泡排序

时间复杂度

$$(n-1) + (n-2) + \dots + 2 + 1 = n(n-1)/2$$

$$O(n^2)$$

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
(n-3) (n-2) comparisons comparisons	11	23	13	21	23	26	32	35	38

```
void bubbleSort (element a[], int n)
{
    element tmp; int i, j;
    for (i=0; i<n-1; i++) {
        for (j=0; j<n-1-i; j++)
            if (a[j+1] < a[j]) {
                tmp = a[j];
                a[j] = a[j+1];
                a[j+1] = tmp;
            }
    }
}
```

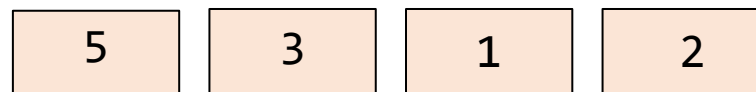
选择排序

❖ Algorithm(算法)

```
for (i=0;i<n;i++){  
    检查a[i]到a[n-1]并假设最小的整数位于a[min]处;  
    Interchange a[i] and a[min];  
}
```

❖ Programming in C(C语言实现)

```
void sort(int a[], int n){  
    int i,j,min,temp;  
    for (i=0;i<n-1;i++) {  
        min=i;  
        for (j=i+1;j<n;j++){  
            if (a[j]<a[min]) min=j;  
        }  
        temp=a[i]; a[i]=a[min]; a[min]=temp;  
    }  
}
```



选择排序案例

	[0]	[1]	[2]	[3]	[4]	[5]
	26	5	37	1	19	24
i=0	1	5	37	26	19	24
i=1	1	5	37	26	19	24
i=2	1	5	19	26	37	24
i=3	1	5	19	24	37	26
i=4	1	5	19	24	26	37

选择排序

时间复杂度

$$(n-1) + (n-2) + \dots + 2 = n(n-1)/2 - 1 \quad O(n^2)$$

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
37	5	46	35	37	46	58	61

```
void selectionSort(int a[], int n)
```

```
{
```

```
    int i,j,min,temp;
```

```
    for (i=0;i<n-1;i++) {
```

```
        min=i;
```

```
        for (j=i+1;j<n;j++){
```

```
            if (a[j]<a[min]) min=j;
```

```
        } min=i
```

```
        temp=a[i]; a[i]=a[min]; a[min]=temp;
```

```
    }
```

```
}
```

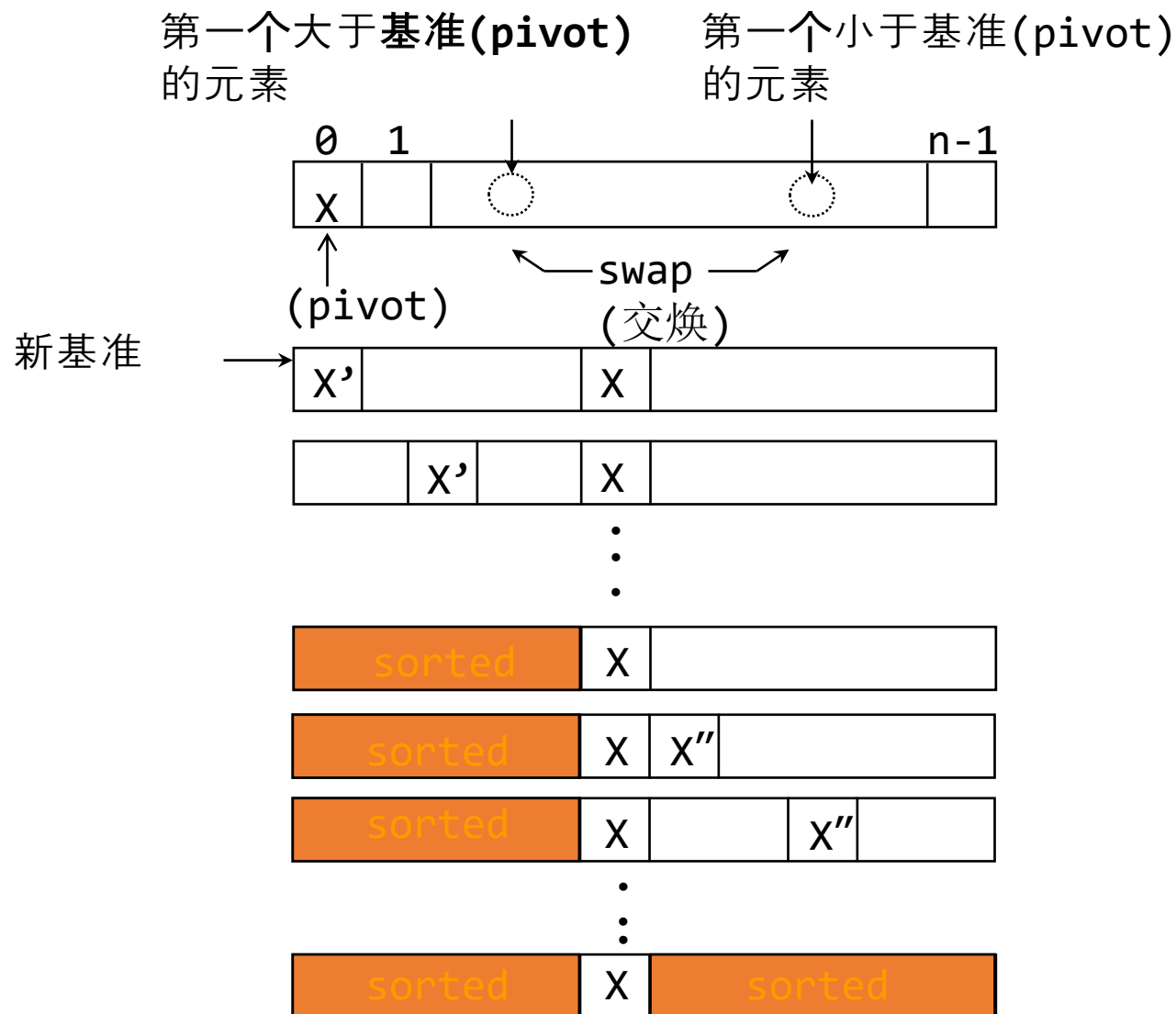
快速排序

❖ 分治法

- 两个阶段
- 分割与处理

❖ 使用递归

- 需要栈来管理递归调用



快速排序案例

[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	left	right
26	5	37	1	61	11	59	15	48	19	1	10
11	5	19	1	15	26	59	61	48	37	1	5
1	5	11	19	15	26	59	61	48	37	1	2
1	5	11	19	15	26	59	61	48	37	4	5
1	5	11	15	19	26	59	61	48	37	7	10
1	5	11	15	19	26	48	37	59	61	7	8
1	5	11	15	19	26	37	48	59	61	10	10
1	5	11	15	19	26	37	48	59	61		

快速排序案例 – 最坏情况

[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	left	right
61	59	48	37	26	19	15	11	5	1	1	10
1	59	48	37	26	19	15	11	5	61	1	9
1	59	48	37	26	19	15	11	5	61	2	9
1	5	48	37	26	19	15	11	59	61	2	8
1	5	48	37	26	19	15	11	59	61	2	7
1	5	11	37	26	19	15	48	59	61	2	6
1	5	11	37	26	19	15	48	59	61	4	7
⋮											

快速排序代码

```
void quicksort (element a[], int left, int right)
{
    int pivot, i, j;    element temp;
    if (left < right) {
        i = left; j = right + 1;
        pivot = a[left].key;
        do {
            do i++; while (a[i].key < pivot); /*&& i<right*/
            do j--; while (a[j].key > pivot); /*&& j>left */
            if (i < j) SWAP(a[i], a[j], temp);
        } while (i < j);
        SWAP (a[left], a[j], temp);
        quicksort(a, left, j - 1);
        quicksort(a, j + 1, right);
    }
}
```

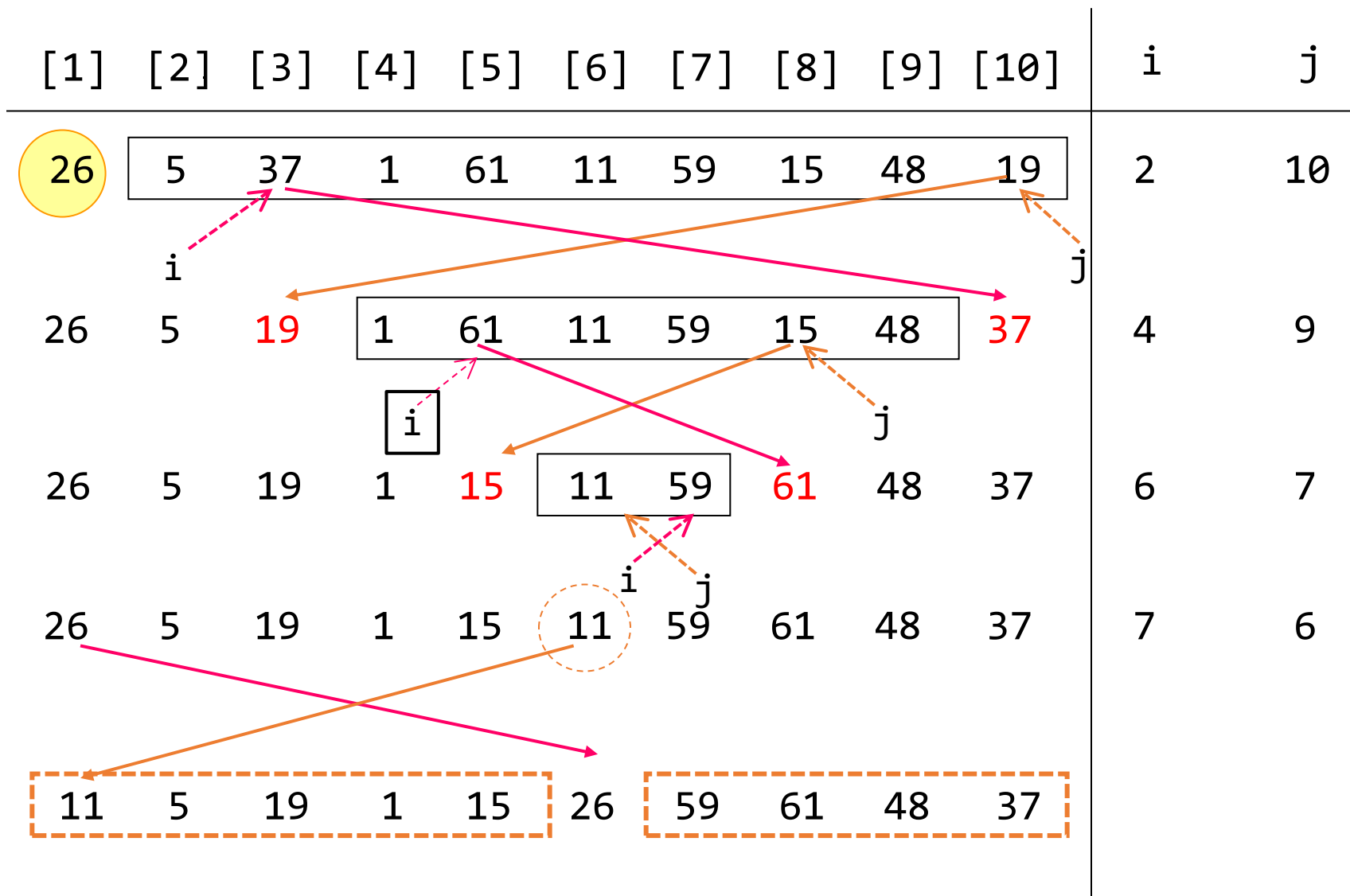
快速排序

Diagram illustrating the array structure and pointers for the merge sort algorithm. The array is divided into two halves: the left half (indices 0 to 5) and the right half (indices 6 to 7). The left half contains the values [32, 26, 23, 13, 35, 7], and the right half contains [38, 35]. The pointers i and j are shown pointing to the middle elements of each half, specifically to index 3 (value 35) in the left half and index 6 (value 38) in the right half. The elements at these positions are highlighted in green.

```
void quicksort (element a[], int left, int right)
{
    = 0 = 7
```

```
int pivot, i, j;    element temp;
if (left < right) {
    i = left; j = right + 1;
    pivot = a[left].key;
    do {
        do i++; while (a[i].key < pivot);
        do j--; while (a[j].key > pivot);
        if (i < j) SWAP(a[i], a[j], temp);
    } while (i < j);
    SWAP (a[left], a[j], temp);
    quicksort(a, left, j - 1);
    quicksort(a, j + 1, right);
}
}
```

快排案例



快速排序分析

- 将列表分割成两个“大小相等”的子列表

* $T(n)$: 排序 n 个记录所需时间

$$\begin{aligned} T(n) &\leq c \cdot n + 2 \cdot T(n/2) \\ &\leq c \cdot n + 2(c \cdot n/2 + 2 \cdot T(n/4)) = 2 \cdot c \cdot n + 4 \cdot T(n/4) \\ &\dots \\ &\leq c \cdot n \cdot \log n + n \cdot T(1) \\ &= O(n \log n) \end{aligned}$$

- 将列表分割成一个含 $n-1$ 个记录和一个含 0 个记录子列表(最坏情况)

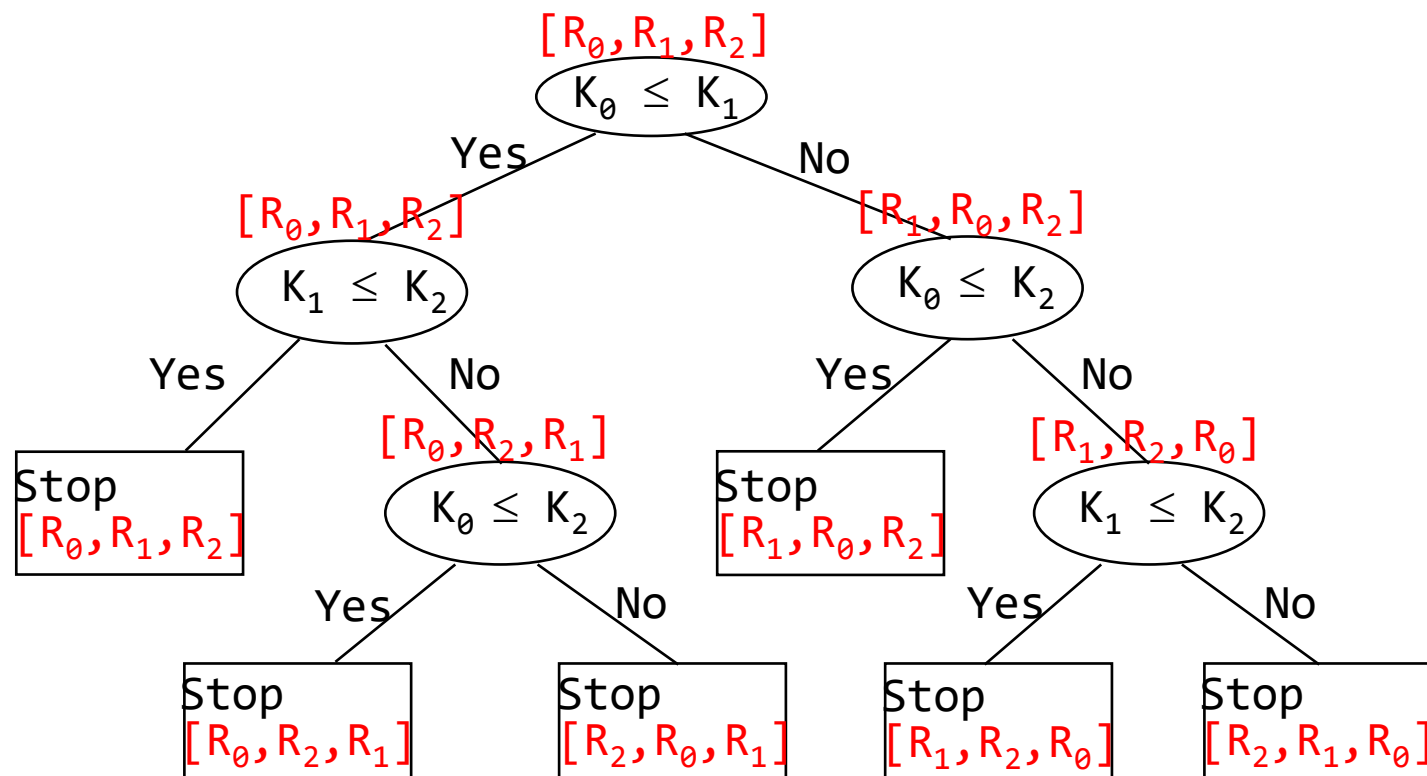
$$\begin{aligned} T(n) &\leq c \cdot n + T(n-1) + T(0) \\ &\leq c \cdot n + c \cdot (n-1) + T(n-2) \\ &\dots \\ &= O(n^2) \end{aligned}$$

优化快速排序

- 快速排序需栈空间实现递归
- 若列表均匀分割，最大递归深度为 $\log n$
 - 空间复杂度 $O(\log n)$
- 最坏情况：每次递归将列表分割为左子列表(大小 $n-1$)和右子列表(大小 0)
 - 空间复杂度 $O(n)$
- 通过首先对较小的子列表进行排序，可以实现栈空间的逐渐减少
 - 空间复杂度 $O(\log n)$
- 基准(pivot)更好的选择是当前子列表中的第一个、中间和最后一个键的中位数

决策树

- 描述排序过程
- 树的每个顶点(vertex)表示一次键比较
- 分支表示比较结果
- 决策树中的一条路径表示算法可能产生的计算序列



Properties of Decision Tree

定理: 任何用于排序 n 个不同元素的决策树, 其高度至少为 $\log(n!) + 1$

→ n 个元素对应 $n!$ 个叶节点

→ 高度为 k 的二叉树叶节点数最多为 2^{k-1}

→ 因此, 决策树高度 $\geq \log(n!) + 1$

推论: 任何仅通过比较进行排序的算法, 其最坏情况计算时间为 $\Omega(n \cdot \log n)$

$$n! = n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot 3 \cdot 2 \cdot 1 \geq (n/2)^{n/2}$$

$$\log(n!) \geq (n/2) \cdot \log(n/2) = \Omega(n \cdot \log n)$$

排序算法的下界: $\Omega(n \cdot \log n)$

归并排序

- ❖ 若 $n < 2$, 数组已排序, 停止
- ❖ 否则, $n > 1$, 按以下步骤执行:
 - 将数组分为左右两半
 - 分别排序左半部分和右半部分
 - 合并已排序的左右两半
- ❖ 时间复杂度
 - $T(n) = 0$, if $n < 2$.
 - $T(n) = T(n/2) + T(n/2) + \Theta(n)$, if $n > 1$. (类似于快速排序)

归并排序案例

[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]
26	5	77	1	61	11	59	15	48	19

26	5	77	1	61	11	59	15	48	19
----	---	----	---	----	----	----	----	----	----

26	5	77	1	61	11	59	15	48	19
----	---	----	---	----	----	----	----	----	----

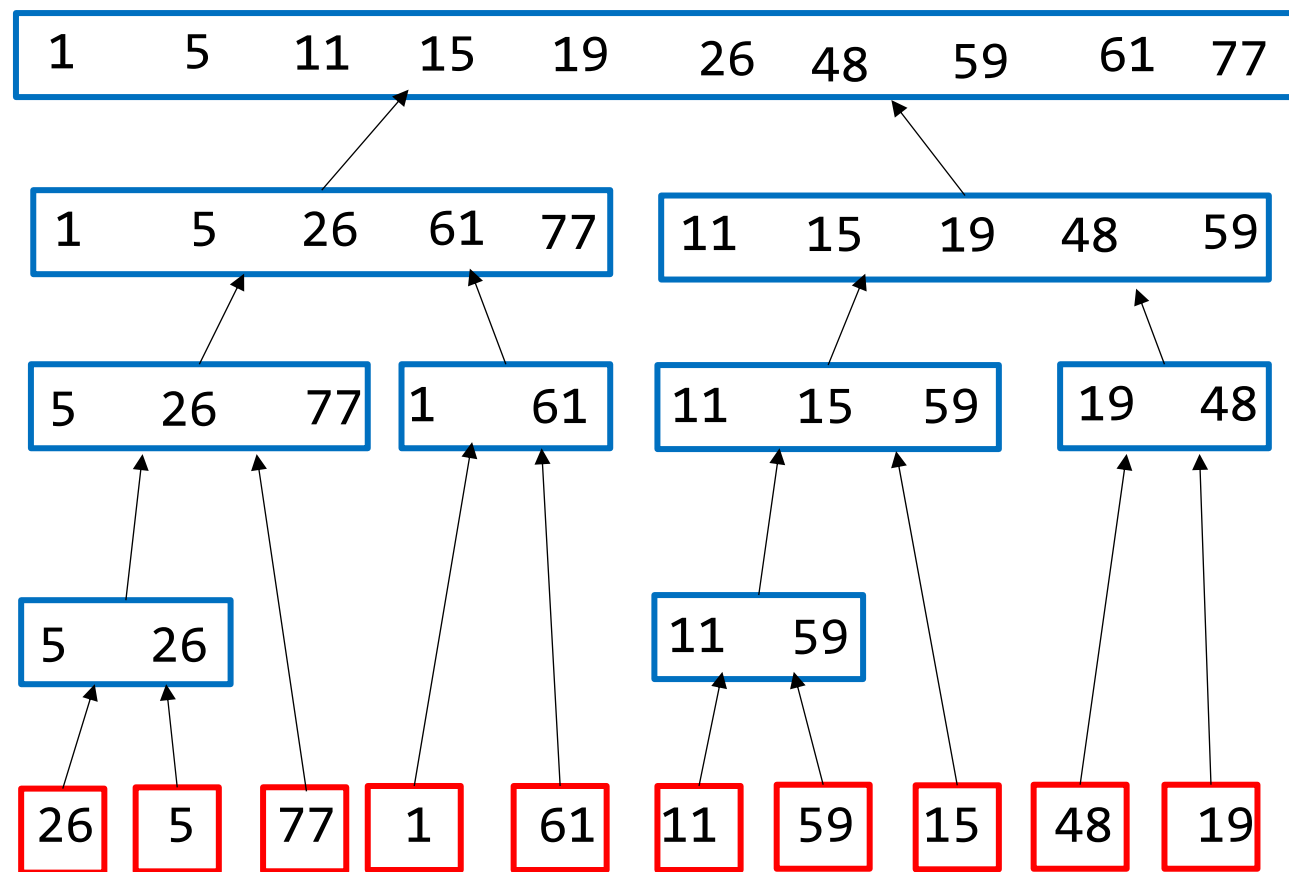
26	5	77	1	61	11	59	15	48	19
----	---	----	---	----	----	----	----	----	----

26	5	77	1	61	11	59	15	48	19
----	---	----	---	----	----	----	----	----	----

Divide
(分割)



归并排序案例



Sort & merge
(排序&合并)

归并(排序)代码

```
void mergeSort(element a[], int left, int right)
{
    /* 待排序数组为 a[left:right] */

    if (left >= right)
        return left;
    int mid = (left + right) / 2 ;
    mergeSort(a, left, mid);           /*排序左半部分*/
    mergeSort(a, mid+1, right);        /*排序右半部分*/
    merge(a, left, mid, right);
}
```


归并排序

```
void mergeSort(element a[], int left, int right)
{
    if (left >= right)
        return left;
    int mid = (left + right) / 2 ;
    mergeSort(a, left, mid);
    mergeSort(a, mid+1, right);
    merge(a, left, mid, right);
}
```

mergeSort(a, 1,10)	mergeSort(a, 1,5)	mergeSort(a, 1,3)	mergeSort(a, 1,2)	mergeSort(a, 1,1)
				mergeSort(a, 2,2)
			mergeSort(a, 3,3)	
		mergeSort(a, 4,5)	mergeSort(a, 4,4)	
			mergeSort(a, 5,5)	
mergeSort(a, 6,10)	mergeSort(a, 6,8)	mergeSort(a, 6,7)	mergeSort(a, 6,6)	
			mergeSort(a, 7,7)	
			mergeSort(a, 8,8)	
	mergeSort(a, 9,10)	mergeSort(a, 9,9)		
		mergeSort(a, 10,10)		

归并排序代码 (1)

```
void merge(element initList[], int i, int m, int n)
{
    int j,k,t;      j = m+1;      k = i;
    while( i <= m && j <= n) {
        if (initList[i].key <= initList[j].key)
            mergeList[k++] = initList[i++];
        else
            mergeList[k++] = initList[j++];
    }
    if (i > m) /* mergedList[k:n] = initList[j:n] */
        for(t = j; t <= n; t++) mergeList[k++] = initList[t];
    else /* mergedList[k:n] = initList[i:m] */
        for(t = i; t <= m; t++)
            mergeList[k++] = initList[t];
    for(t = i; t<=n; t++)
        initList[t] = mergeList[t];
}
```

element mergeList[MAX_SIZE] /* 全局变量 */

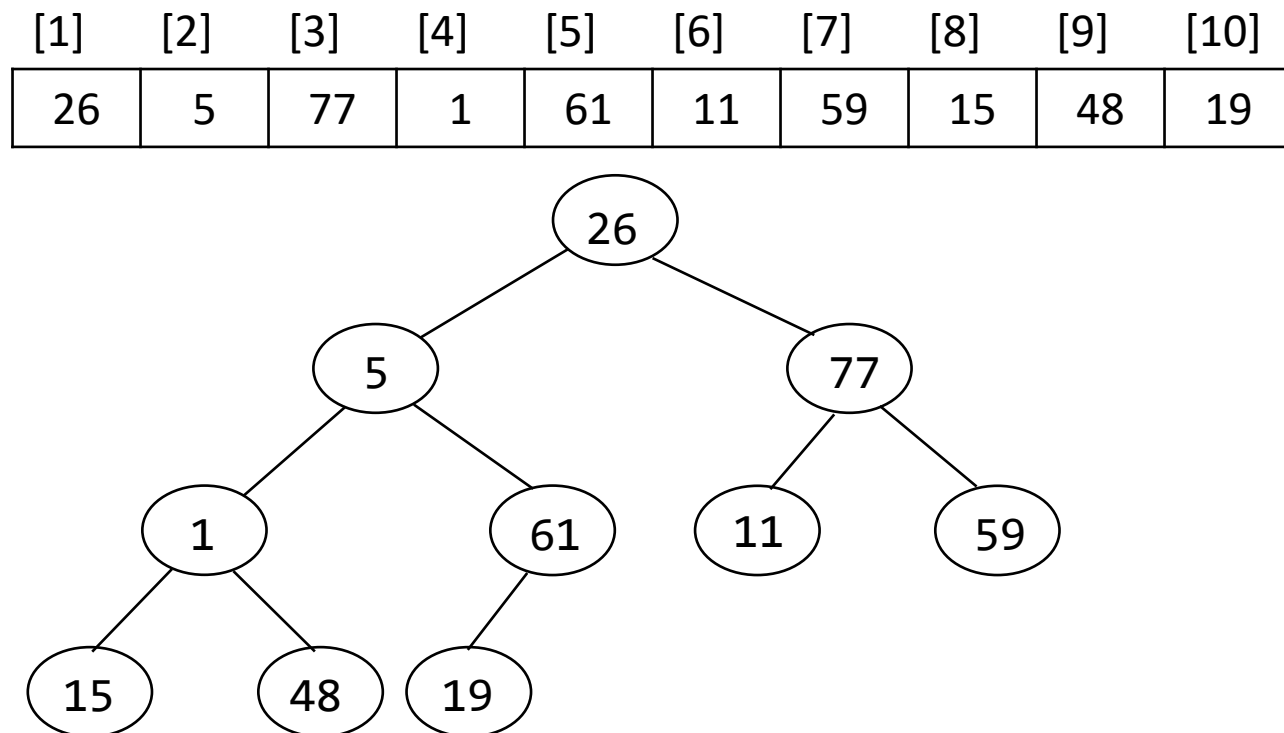
堆排序

❖ 利用最大堆结构

- 平均情况： $O(n \log n)$
- 最坏情况： $O(n \log n)$

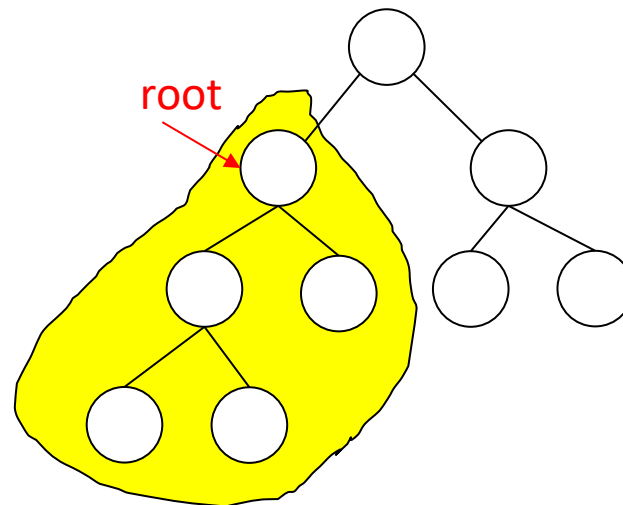
❖ 调整二叉树以构建最大堆

→ $O(d)$, d : 树的深度



堆排序代码 (1)

```
void adjust(element list[], int root, int n)
{  int child, rootkey;
   element temp;
   temp = list[root];
   rootkey = list[root].key;
   child = 2 * root; /* left child */
   while(child <= n){
       if (child < n && list[child].key < list[child+1].key)
           child++;
       if (rootkey > list[child].key)
           break;
       else {
           list[child/2] = list[child];
           child *= 2;
       } /* else */
   } /* while */
   list[child/2] = temp;
}
```



堆排序代码 (2)

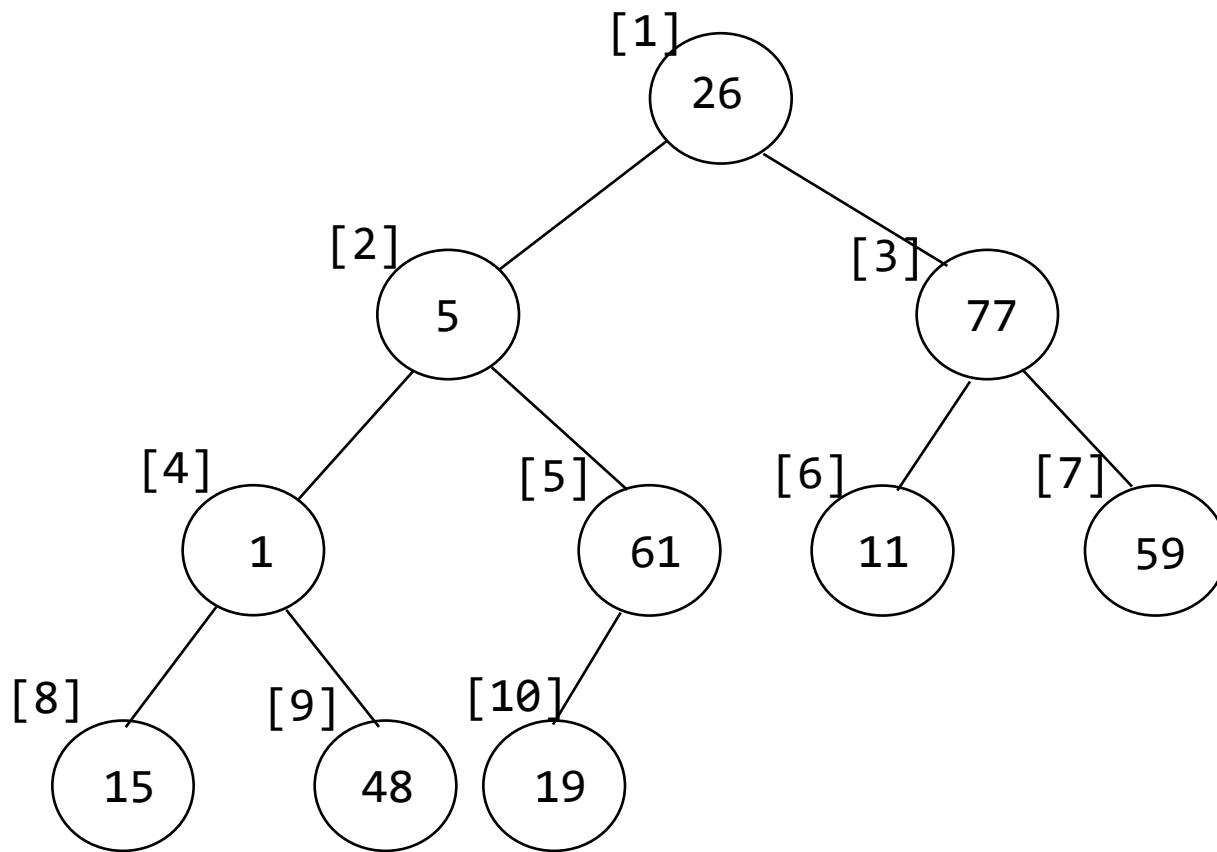
```
void heapsort(element list[], int n)
{ /* perform a heapsort on the array */
    int i, j;
    element temp;
    for (i = n/2; i > 0; i--)
        adjust(list, i, n);          /* 初始堆构建 */
    for (i = n - 1; i > 0; i--)      /* 堆调整 */
    {
        SWAP(list[1], list[i+1], temp);
        adjust(list, 1, i);
    }
}
```

堆排序案例 (1)

Input list : (26, 5, 77, 1, 61, 11, 59, 15, 48, 19)

Output list: (1, 5, 11, 15, 19, 26, 48, 59, 61, 77)

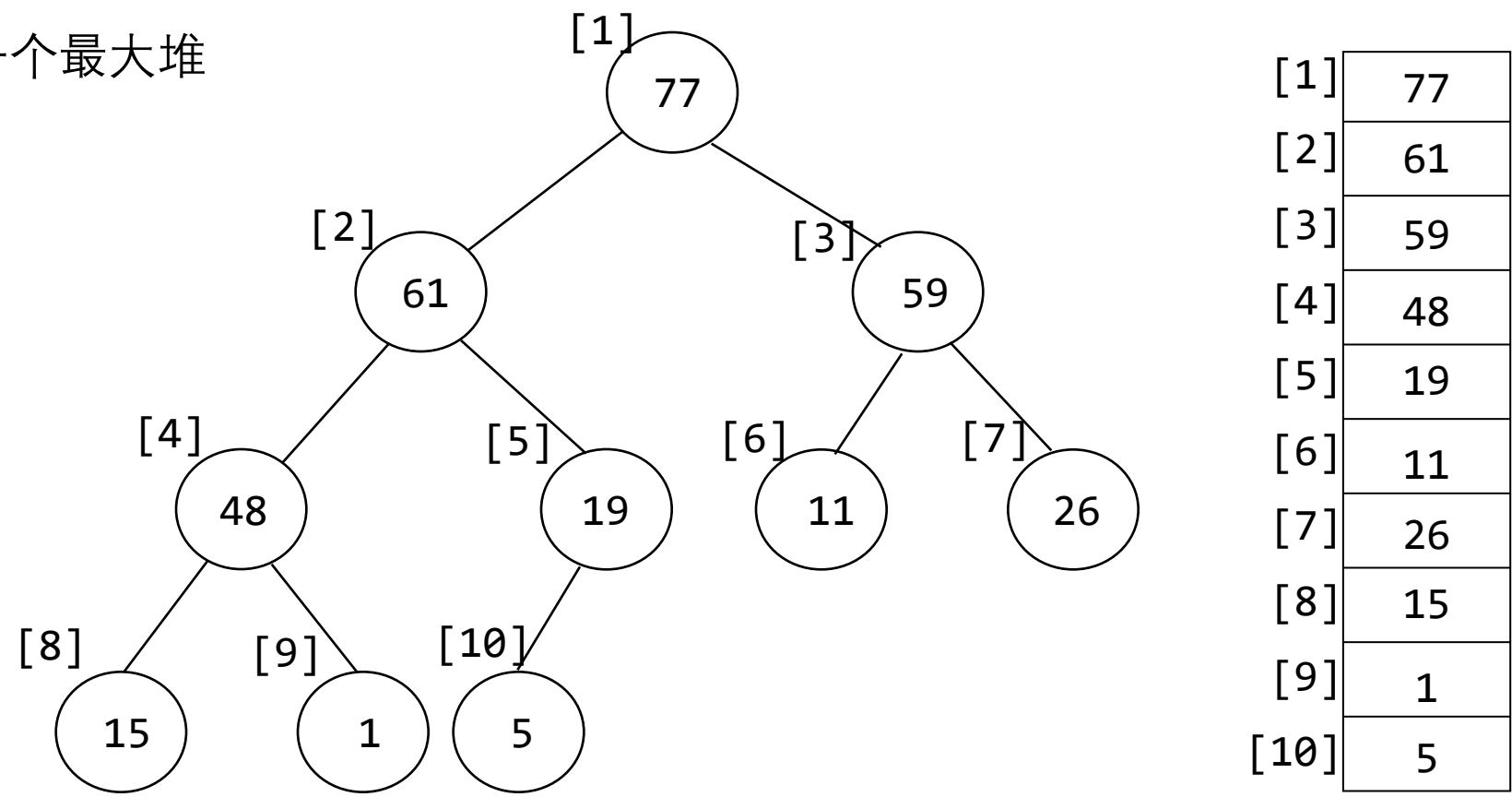
Initial state



[1]	26
[2]	5
[3]	77
[4]	1
[5]	61
[6]	11
[7]	59
[8]	15
[9]	48
[10]	19

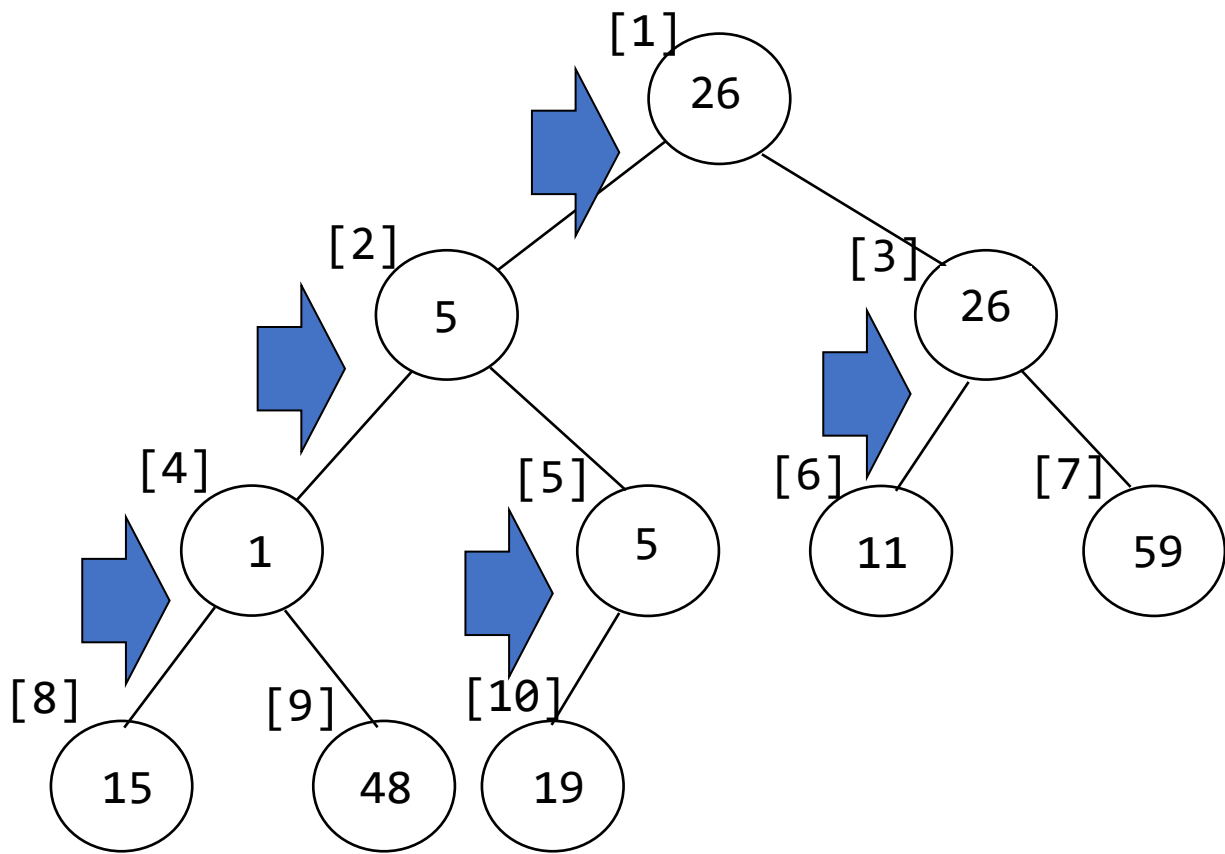
堆排序案例 (2)

第 1 步: 构建一个最大堆



初始最大堆调整

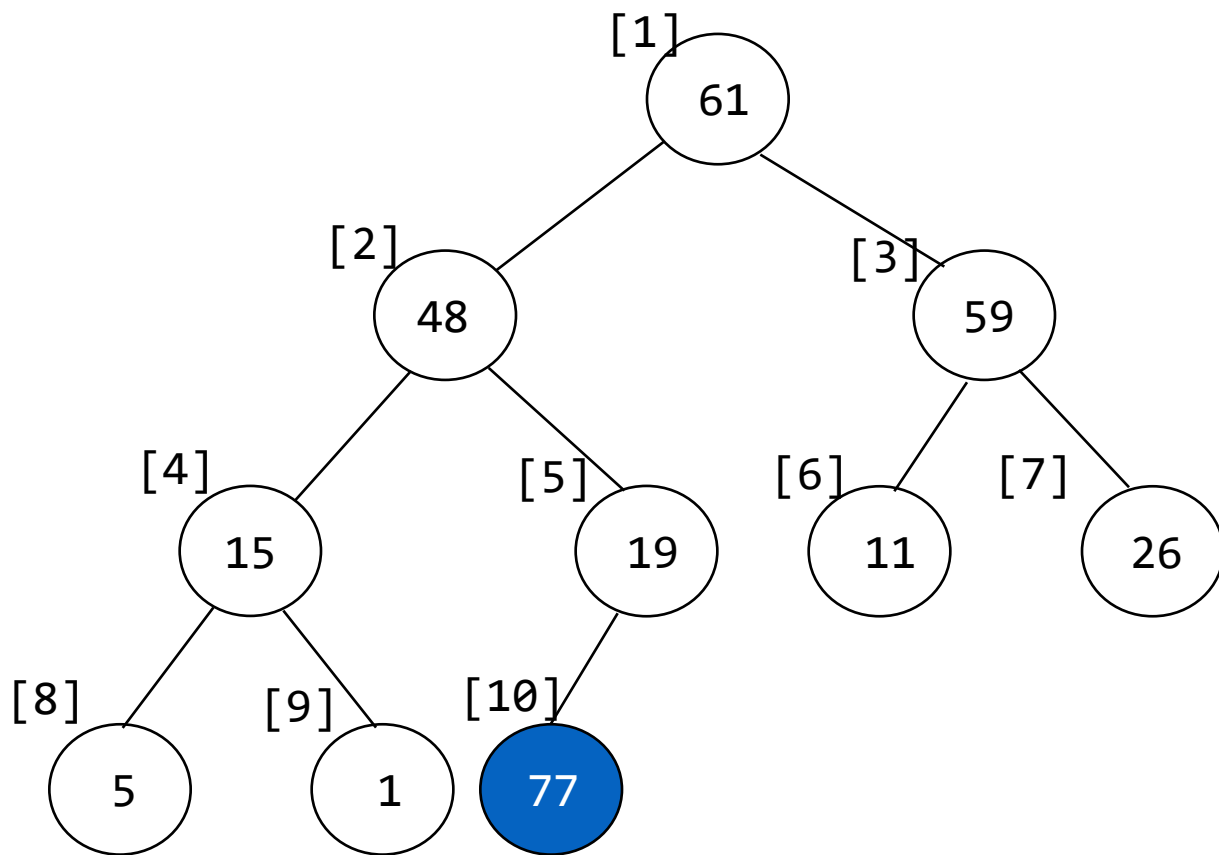
```
for (i = n/2; i > 0; i--)  
    adjust(list, i, n);
```



```
void adjust(element list[], int root, int n)  
{  
    int child, rootkey;  
    element temp;  
    temp = list[root];  
    rootkey = list[root].key;  
    child = 2 * root; /* left child */  
    while(child <= n){  
        if (child < n &&  
            list[child].key < list[child+1].key)  
            child++;  
        if (rootkey > list[child].key)  
            break;  
        else {  
            list[child/2] = list[child];  
            child *= 2;  
        } /* else */  
    } /* while */  
    list[child/2] = temp;  
}
```

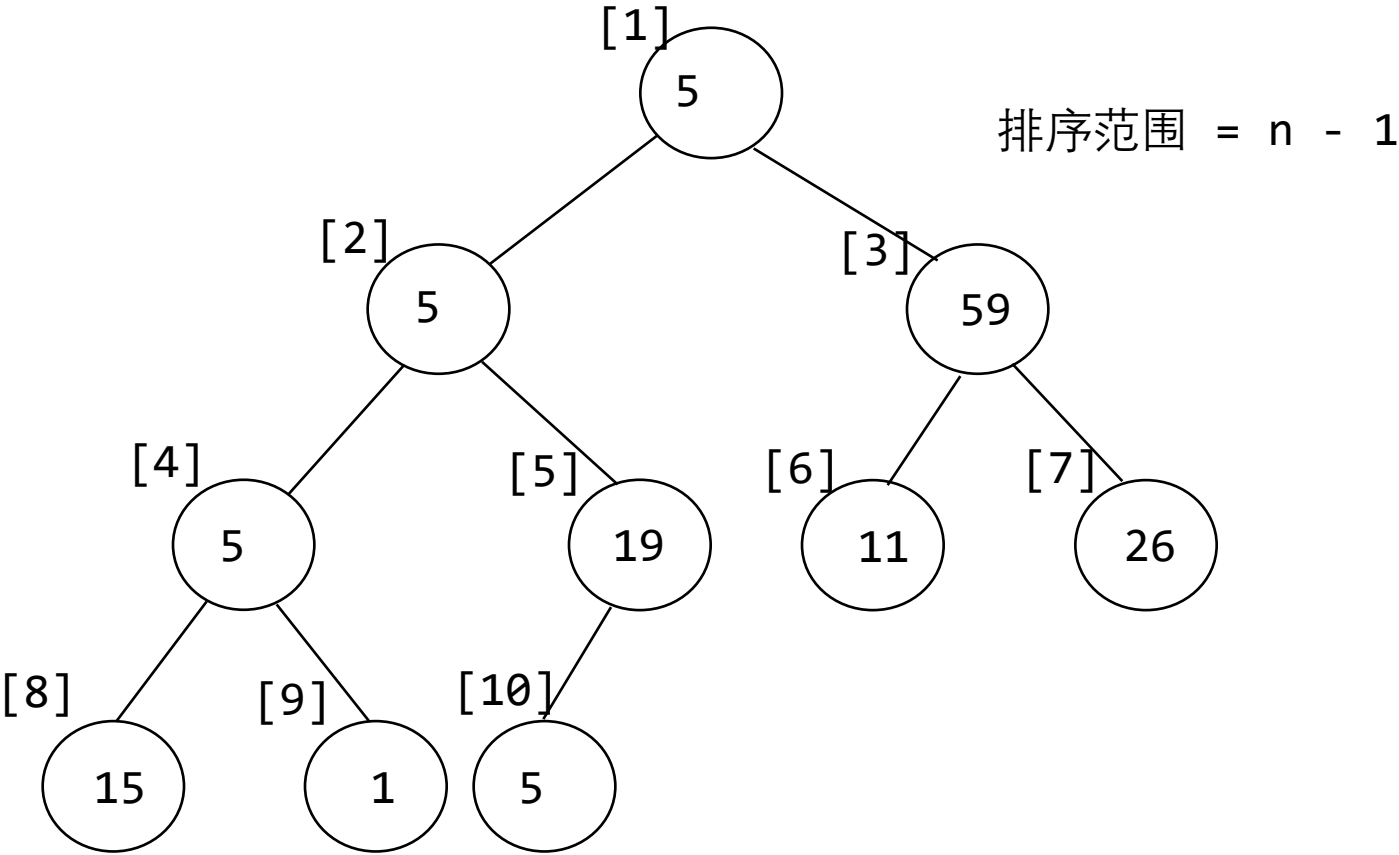

堆排序案例 (3)

第 2 步: 交换根节点与最后一个节点

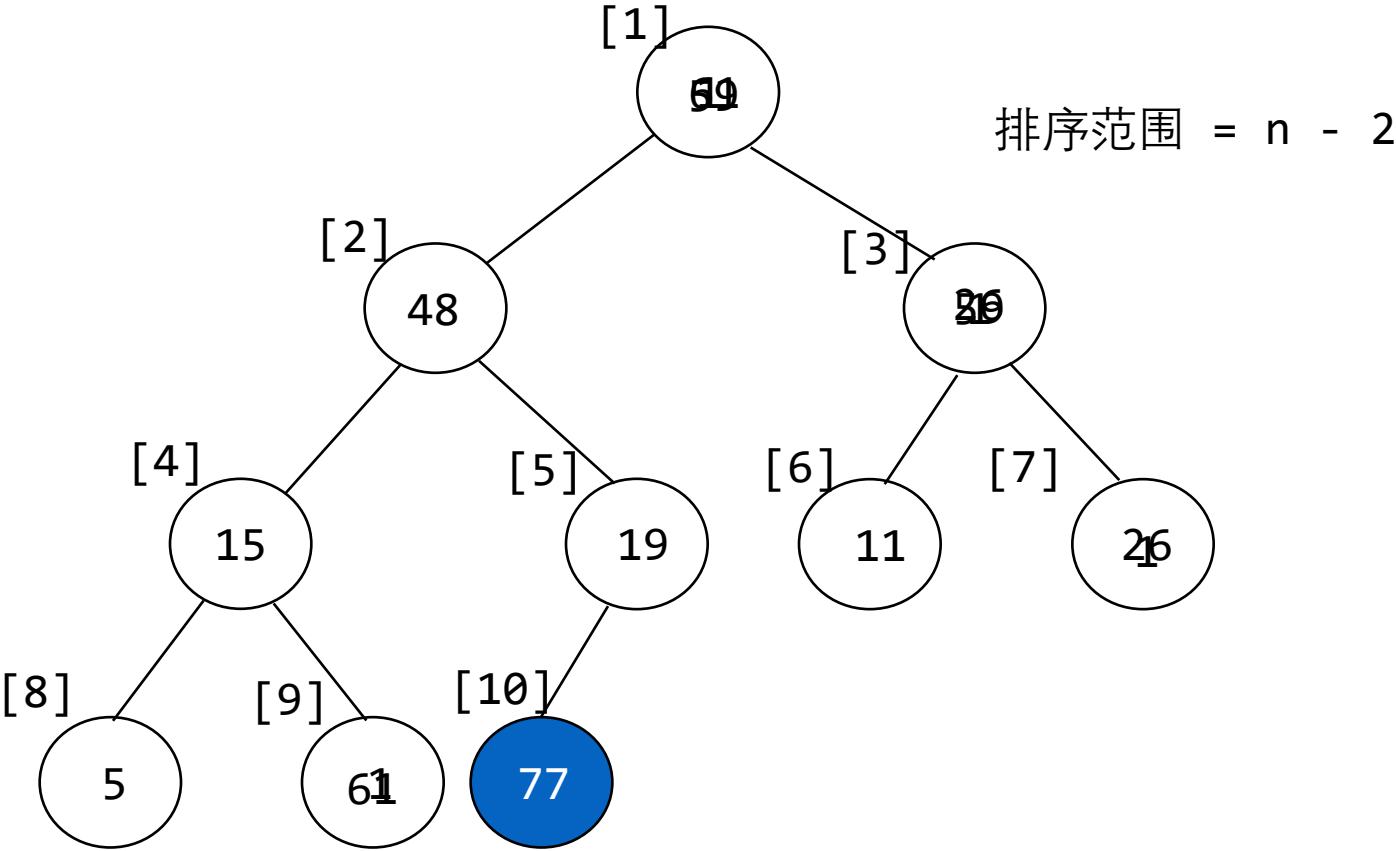


[1]	61
[2]	48
[3]	59
[4]	15
[5]	19
[6]	11
[7]	26
[8]	5
[9]	1
[10]	77

堆排序 – 第 2 步

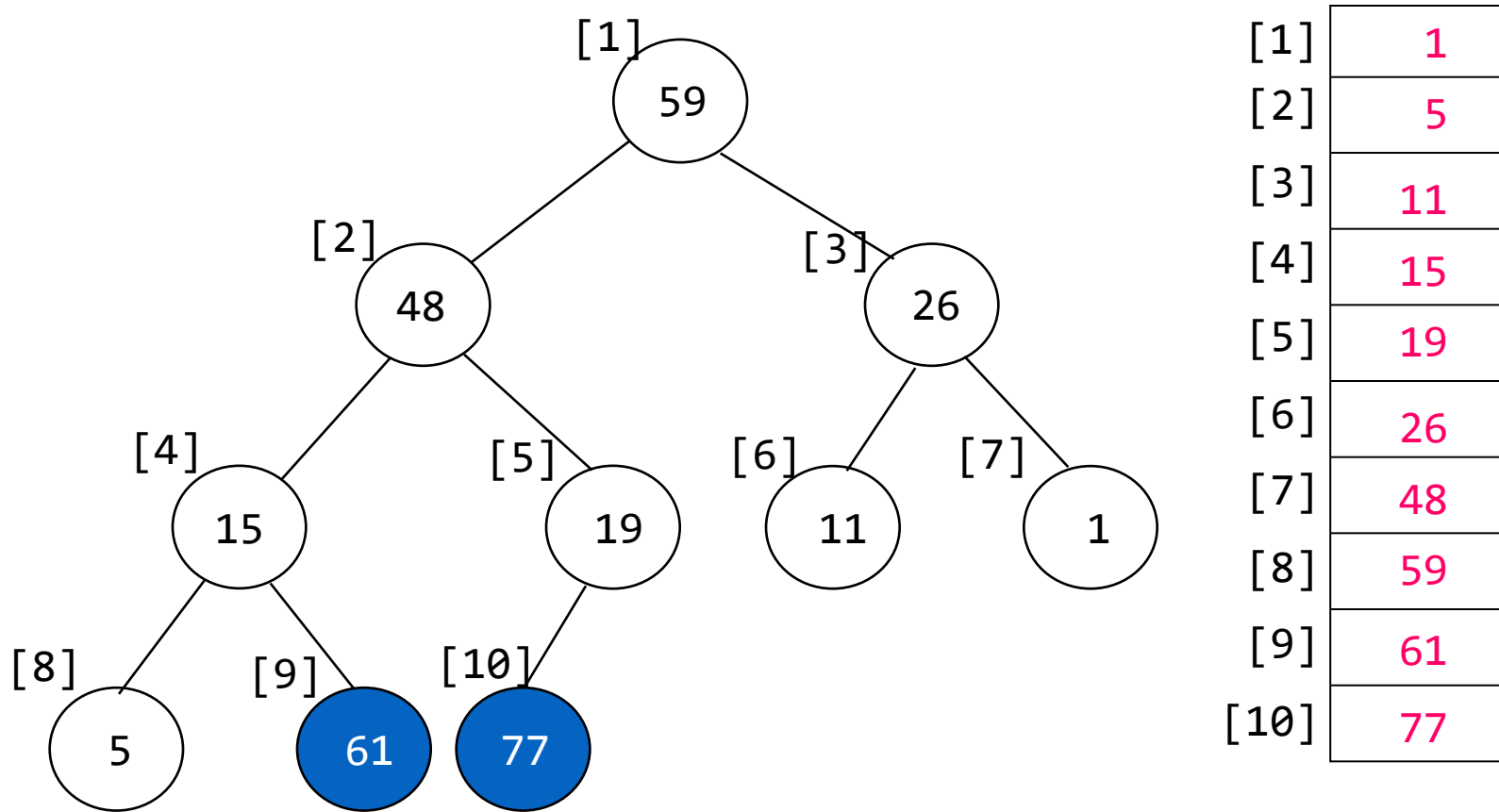


堆排序 – 第 3 步



堆排序案例 (4)

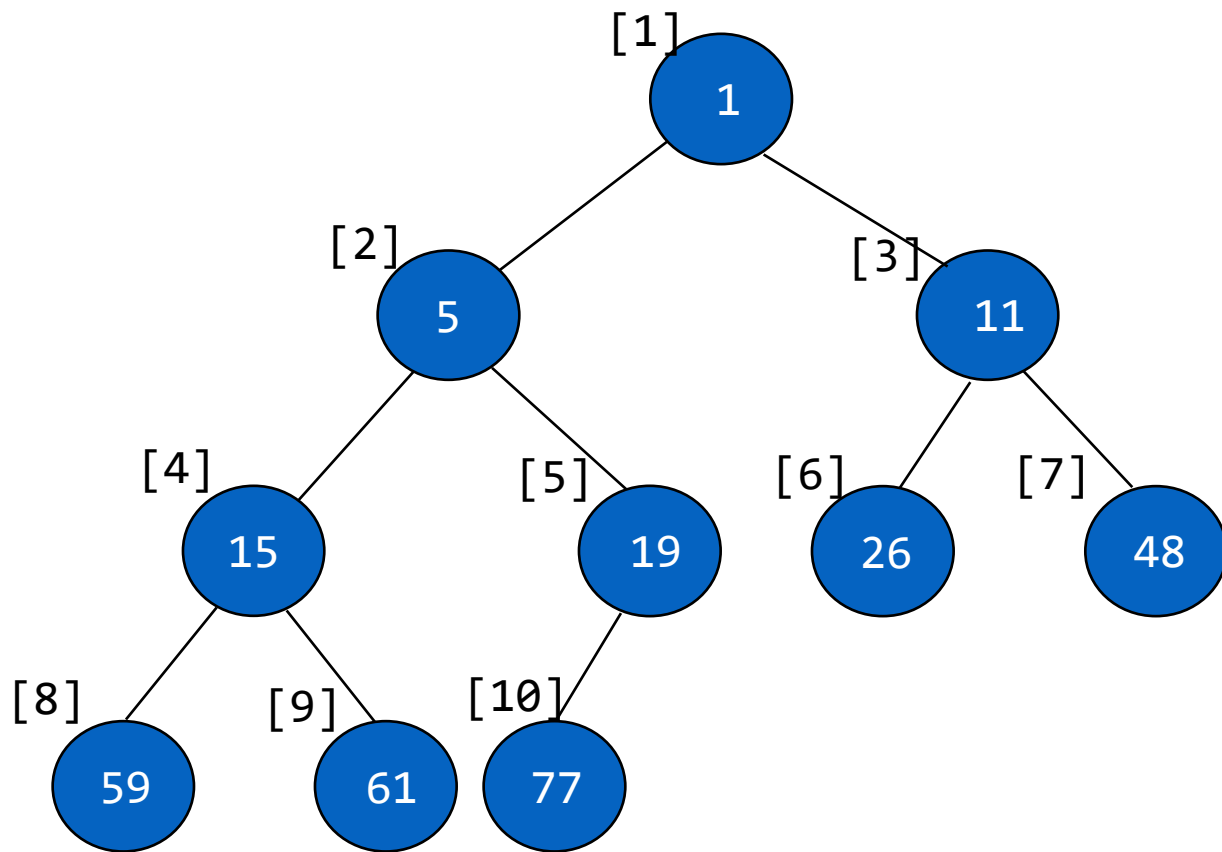
第 3 步: 将根节点与倒数第二个节点交换。调整堆, 但不包括已排序的最后两个节点。



重复上述步骤直到只剩一个节点

堆排序案例 (5)

最后一步



[1]	1
[2]	5
[3]	11
[4]	15
[5]	19
[6]	26
[7]	48
[8]	59
[9]	61
[10]	77

内部排序总结 (1)

❖ 插入排序

- 当列表已经部分排序时效果较好
- 对于"小规模"数据量 n 是最佳的排序方法

❖ 归并排序

- 最佳的最坏情况性能，但需要比堆排序更多的存储空间

❖ 快速排序

- 最佳的平均情况性能
- 最坏情况性能较差，即 $O(n^2)$

内部排序总结 (2)

(排序)方法	最坏(情况)	平均(情况)
Insertion sort(插入排序)	n^2	n^2
Heap sort(堆排序)	$n \log n$	$n \log n$
Merge sort(归并排序)	$n \log n$	$n \log n$
Quick sort(快速排序)	n^2	$n \log n$

